

Assignment 03 Extra

Digital RTL Design

Sequential Circuit Design

Marwan Abdelfattah

Contents

1	Question 1	2
2	Question 2	2
2.1	4-Bit Synchronous Counter with Asynchronous Active-Low Set	2
2.1.1	Inputs	2
2.1.2	Output	2
2.2	RTL Design	2
2.3	Testbench	3
2.4	Simulation Waveform	4
3	Question 3	5
3.1	4-Bit Synchronous Counter with Asynchronous Active-Low Set	5
3.1.1	Inputs	5
3.1.2	Output	5
3.2	RTL Design	5
3.3	Testbench	6
3.4	Simulation Waveform	7
4	Question 4	8
4.1	2-Bit Gray Counter	8
4.1.1	Inputs	8
4.1.2	Output	8
4.1.3	Testbench Requirements	8
4.2	RTL Design	9
4.3	Testbench	9
4.4	Simulation Waveform	10

1 Question 1

The completed schematics will be submitted separately as a **PDF file** and are not included in this report.

2 Question 2

2.1 4-Bit Synchronous Counter with Asynchronous Active-Low Set

Implement a **4-bit synchronous counter** using **behavioral modeling**. The counter should increment sequentially from 0 to 15.

The design should include an **asynchronous active-low set** input that sets all counter bits to 1.

2.1.1 Inputs

- **clk**: Clock input.
- **set**: Asynchronous active-low set input that sets all counter bits to 1.

2.1.2 Output

- **out**: 4-bit counter output.

The counter should operate synchronously with the clock and increment from:

$$0 \rightarrow 1 \rightarrow 2 \rightarrow \dots \rightarrow 15$$

After reaching 15, the counter should wrap around to 0.

Use the **4-bit ripple counter implemented in Assignment 3** as a **golden reference model**.

Create a **self-checking testbench** for the synchronous counter. The testbench should instantiate both the newly implemented synchronous counter and the ripple counter golden model and compare their outputs to verify that the two designs behave identically.

2.2 RTL Design

```
1 module counter (  
2     input clk, set,  
3     output reg [3:0] out  
4 );  
5 always @(posedge clk or negedge set) begin  
6     if(!set) begin  
7         out <= 4'b1111;  
8     end else begin  
9         out <= out + 1;  
10    end  
11 end  
12 endmodule
```

2.3 Testbench

```
1 module counter_tb ();
2 reg clk, rstn;
3 wire [3:0] out_struct, out_beh;
4
5 initial begin
6     clk = 0;
7     forever begin
8         #1 clk = ~clk;
9     end
10 end
11
12 ripple_counter RIPPLE(clk, rstn, out_struct);
13 counter DUT (clk, rstn, out_beh);
14
15 integer i;
16 initial begin
17
18     rstn = 0;
19     @(negedge clk);
20     rstn = 1;
21
22     if(out_struct != out_beh) begin
23         $display("Reset Function Error!");
24         $stop;
25     end
26
27     for(i=0;i<100;i=i+1) begin
28         @(negedge clk);
29         if(out_struct != out_beh) begin
30             $display("Error!!");
31             $stop;
32         end
33     end
34     $stop;
35 end
36 initial begin
37     $monitor("rstn=%b, clk=%b, out_struct=%b, out_beh=%b",
38             rstn, clk, out_struct, out_beh);
39 end
40 endmodule
```

2.4 Simulation Waveform

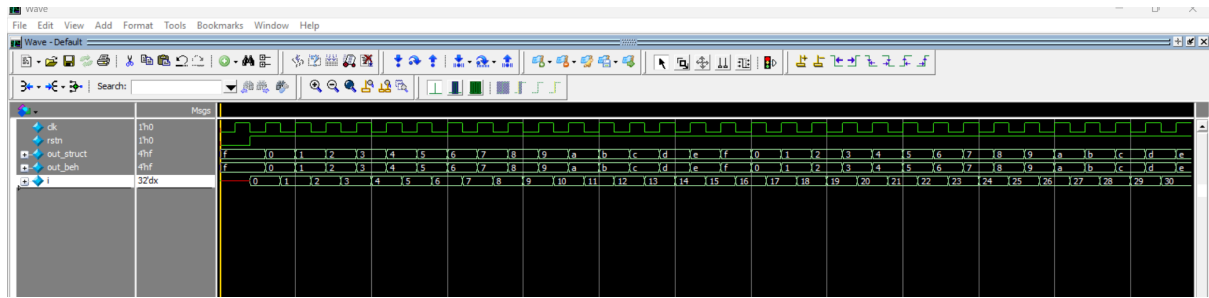


Figure 1: Simulation waveform

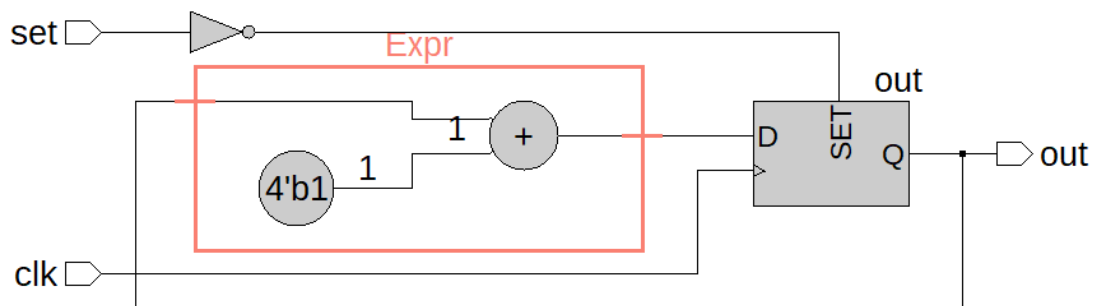


Figure 2: Schematic Diagram

3 Question 3

3.1 4-Bit Synchronous Counter with Asynchronous Active-Low Set

Implement a **4-bit synchronous counter** using **behavioral modeling**. The counter should increment sequentially from 0 to 15.

The design should include an **asynchronous active-low set** input that sets all counter bits to 1.

3.1.1 Inputs

- **clk**: Clock input.
- **set**: Asynchronous active-low set input that sets all counter bits to 1.

3.1.2 Output

- **out**: 4-bit counter output.

The counter should operate synchronously with the clock and increment from:

$$0 \rightarrow 1 \rightarrow 2 \rightarrow \dots \rightarrow 15$$

After reaching 15, the counter should wrap around to 0.

Use the **4-bit ripple counter implemented in Assignment 3** as a **golden reference model**.

Create a **self-checking testbench** for the synchronous counter. The testbench should instantiate both the newly implemented synchronous counter and the ripple counter golden model and compare their outputs to verify that the two designs behave identically.

3.2 RTL Design

```
1 module counter (  
2     input clk, set,  
3     output reg [3:0] out  
4 );  
5 always @(posedge clk or negedge set) begin  
6     if(!set) begin  
7         out <= 4'b1111;  
8     end else begin  
9         out <= out + 1;  
10    end  
11 end  
12 endmodule
```

3.3 Testbench

```
1 module counter_tb ();
2 reg clk, rstn;
3 wire [3:0] out_struct, out_beh;
4
5 initial begin
6     clk = 0;
7     forever begin
8         #1 clk = ~clk;
9     end
10 end
11
12 ripple_counter RIPPLE(clk, rstn, out_struct);
13 counter DUT (clk, rstn, out_beh);
14
15 integer i;
16 initial begin
17
18     rstn = 0;
19     @(negedge clk);
20     rstn = 1;
21
22     if(out_struct != out_beh) begin
23         $display("Reset Function Error!");
24         $stop;
25     end
26
27     for(i=0;i<100;i=i+1) begin
28         @(negedge clk);
29         if(out_struct != out_beh) begin
30             $display("Error!!");
31             $stop;
32         end
33     end
34     $stop;
35 end
36 initial begin
37     $monitor("rstn=%b, clk=%b, out_struct=%b, out_beh=%b",
38             rstn, clk, out_struct, out_beh);
39 end
40 endmodule
```

3.4 Simulation Waveform

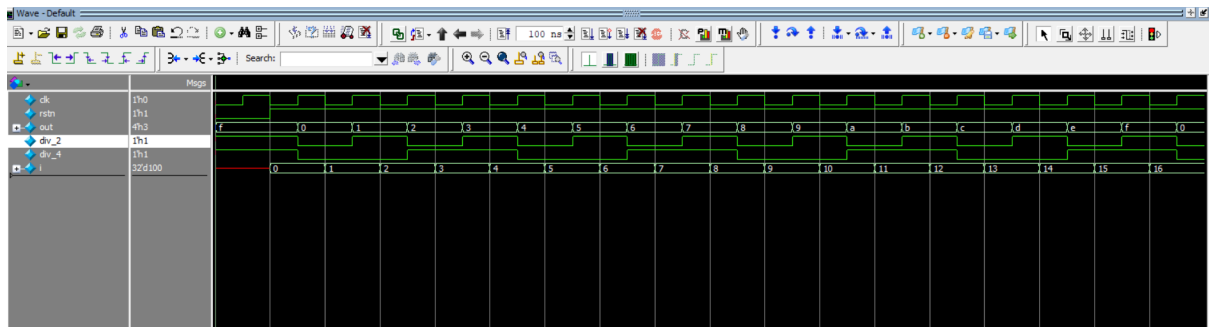


Figure 3: Simulation waveform

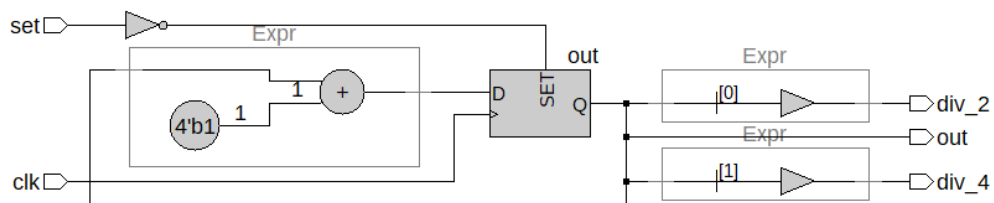


Figure 4: Schematic Diagram

4 Question 4

4.1 2-Bit Gray Counter

Implement a **2-bit Gray counter** according to the following specifications.

4.1.1 Inputs

- `clk`: Clock input.
- `rst`: Reset input.

4.1.2 Output

- `gray_out`: 2-bit Gray-code output.

The design should contain an internal 2-bit binary counter that counts sequentially through:

$$0 \rightarrow 1 \rightarrow 2 \rightarrow 3$$

The binary counter should then wrap around and repeat the sequence.

The Gray-code output should be generated using the relationship between the binary counter and Gray code.

For a 2-bit binary counter represented as:

$$B_1 B_0$$

the corresponding Gray-code output should be:

$$G_1 = B_1$$

$$G_0 = B_1 \oplus B_0$$

Therefore, the most significant bit of the Gray output is the same as the most significant bit of the binary counter, while the least significant Gray-code bit is the XOR of the two binary counter bits.

4.1.3 Testbench Requirements

Create a testbench that verifies the following:

1. Apply the reset signal and verify that `gray_out` is forced to zero.
2. Remove the reset signal.
3. Verify the Gray-code counting pattern using the simulation waveform.

The expected Gray-code sequence should be verified as the binary counter progresses through its four states.

4.2 RTL Design

```
1 module gray_counter (  
2     input clk,  
3     rst,  
4     output [1:0] gray_out  
5 );  
6  
7     reg [1:0] binary_counter;  
8  
9     always @(posedge clk or posedge rst) begin  
10         if (rst) begin  
11             binary_counter <= 0;  
12         end else begin  
13             binary_counter <= binary_counter + 1;  
14         end  
15     end  
16     assign gray_out[1] = binary_counter[1];  
17     assign gray_out[0] = ^binary_counter;  
18 endmodule
```

4.3 Testbench

```
1 module gray_counter_tb ();  
2     reg clk;  
3     reg rst;  
4     wire [1:0] gray_out;  
5  
6     gray_counter DUT (.*);  
7  
8     initial begin  
9         clk = 0;  
10        forever begin  
11            #1 clk = ~clk;  
12        end  
13    end  
14  
15    initial begin  
16        rst = 1;  
17        @(negedge clk);  
18        rst = 0;  
19        if (gray_out != 0) begin  
20            $display("Reset Failed!!");  
21            $stop;  
22        end else begin  
23            repeat (20) @(negedge clk);  
24        end  
25        $stop;  
26    end  
27 endmodule
```

4.4 Simulation Waveform

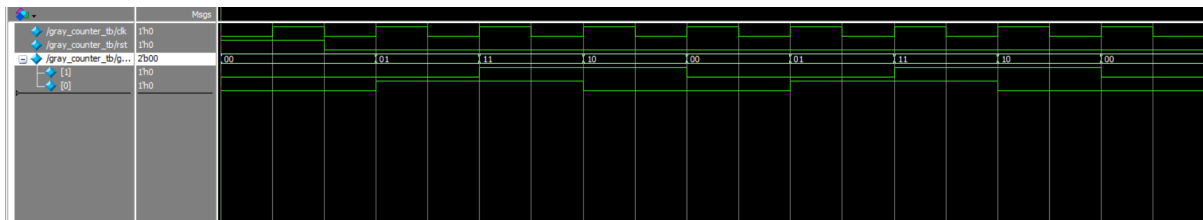


Figure 5: Simulation waveform

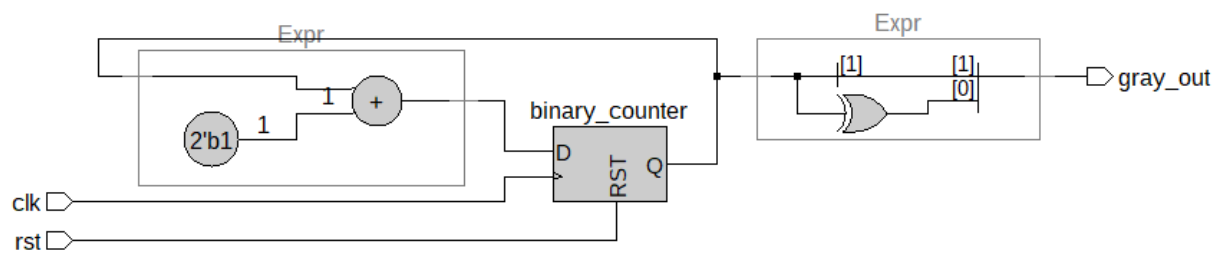


Figure 6: Schematic Diagram